智能摄像机的自动聚焦算法

> 时间限制：10s（每用例） 空间限制：512MB

背景信息

在智能视频监控的世界里，每一帧画面都必须清晰锐利。自动聚焦（AF）是其中无声的英雄——也可能是隐蔽的瓶颈。随着场景越来越复杂，对更快、更可靠聚焦的需求从未如此强烈。在本赛中，你将进入智能摄像机的镜头，设计一个自动聚焦算法，用最少的"寻峰"动作找到完美焦点。调节镜头，驾驭清晰度曲线，为明天的智慧城市打造视觉引擎。

视频监控摄像机镜头分为**定焦**与**变焦**两类。变焦镜头依赖自动聚焦（AF）算法驱动聚焦电机移动，直到画面清晰。随着监控场景向更暗、更繁忙、更复杂的环境扩展，市场对聚焦成功率和聚焦速度的要求越来越高，聚焦体验已成为产品竞争力的关键差异点。

对比度式自动聚焦最常用的方法是**"爬山"算法**：电机逐步移动，在每个位置评估图像清晰度，利用清晰度的变化定位峰值——即画面最清晰的电机位置。理论上，扫描整个电机行程必然能找到全局峰值；但实际应用中聚焦时间严格受限，算法必须在尽可能短的时间内收敛到清晰度峰值，同时避免频繁的电机方向反转（反转会造成可见的画面振荡，损害用户体验）。

难点：真实世界场景

清晰度曲线很少是干净的单峰。真实监控画面会引入多种干扰，容易误导朴素的爬山搜索：

- **低照度（噪声）场景**。夜间户外场景（如乡间小路）照度可能低至星光级（约 0.09 lux），图像充满传感器噪声，清晰度曲线上产生大量**假峰**，干扰真实峰值的搜索。
- **运动物体场景**。繁忙路口行人、车辆不断穿越画面，运动物体同样会在清晰度曲线上制造**假峰**。
- **前景/背景场景**。依安装位置不同，画面可能包含明显分离的前景与背景物体，使清晰度曲线呈现明显的**双峰**。算法必须按约定规则落在合适的聚焦位置上。

题目描述

给定一个**清晰度数组**。数组每个元素是一个非负整数，表示聚焦电机停在该下标时图像的清晰度，数值越大表示画面越清晰。数组下标从 `0` 到 `length - 1`，电机初始停在下标 `0`。

你的任务是设计搜索算法**找到清晰度最大值所在位置——"主峰"**，并让电机最终停在该下标上。搜索过程需按优先级满足以下目标：

1. **最小化聚焦振荡**。搜索过程中方向反转次数越少越好。
2. **最小化搜索时间**。搜索收敛越快越好。
3. **最小化冗余步数**。搜索越过主峰继续向右探的距离越短越好。

关键定义

- **搜索步**。算法每读取数组中的一个值计为**一步**。读取的下标可以不连续。
- **搜索代价**。聚焦动作由步进电机物理驱动，电机从当前位置移动到另一位置有真实代价。单次读取的**搜索代价**是当前下标与上一次读取下标之差的绝对值。电机初始在 `0`，因此第一次读取的代价等于其下标本身。
- **搜索时间**。**搜索时间** = 所有搜索步之和 + 所有搜索代价之和（模拟电机总移动时间）。
- **聚焦振荡**。移动方向每次发生改变——即当前读取的移动方向与上一次相反——计为**一次振荡**。第一次读取不计振荡。
- **冗余步数**。**冗余步数** = 搜索曾到达的最右下标 − 主峰下标（不为负）。

平局规则

若数组中存在多个最大清晰度值，则取**最接近数组中间 `(length-1)/2` 的位置**作为目标（"主峰"）；距离相同取更小下标。例如 `[6, 8, 10, 9, 8, 10, 4, 7, 10, 4]` 中三个最大值分别在下标 `2`、`5`、`8`，其中 `5` 最接近中间，因此目标为下标 `5`。

交互协议

本题为**交互式判题**：判题器（checker）启动你的程序，通过标准输入/标准输出逐条完成查询，模拟电机运行并自动统计搜索步、搜索代价、振荡次数与冗余步数。**协议中的下标一律为 0-based**。

流程如下：

```
判题器 → 程序：    一行整数 N（数组长度）
程序   → 判题器：  一行整数 idx（0 ≤ idx < N，查询下标）
判题器 → 程序：    一行整数（该下标的清晰度值）
程序   → 判题器：  一行整数 idx   （重复查询…）
...
程序   → 判题器：  一行 "DONE"（搜索结束）
程序               退出且退出码为 0
```

要求：

- 所有协议行均为 ASCII 文本，以 `\n`（或 `\r\n`）结尾，**单行内容不超过 64 字节**。
- 程序每次查询输出一行下标（0-based），读取一行清晰度值。

- 查询下标必须在 `[0, N-1]` 内，否则判为协议错误。
- 搜索结束必须输出 `DONE` 并正常退出（退出码 0）。
- **除协议行外，不得向标准输出写任何其他内容**（调试信息请写到标准错误）。
- 输出必须及时刷新（C/C++ 用 `setvbuf(stdout, NULL, _IONBF, 0)` 或每次 `fflush`；Java 每次 `System.out.println` 后 `System.out.flush()`）。

注意：本题没有预置框架函数。你需要自己实现整个程序：读取 N、按上述协议逐条查询清晰度值、最后输出 `DONE`。最终电机停在下标即最后输出的下标（最后一次查询的 idx）；`DONE` 本身不携带下标。

提交要求

- 提交**单个源文件**，文件名必须是 `main.c`（C）或 `main.cpp`（C++）或 `Main.java`（Java）。
- 编译命令：`gcc -O3 -Wall -std=c17` / `g++ -O3 -Wall -std=c++17` / `javac`。
- 程序启动后从标准输入读取，向标准输出写入，退出码为 0 表示成功。

评分规则

判题器自动统计每次查询的电机移动，最终按下式计算单用例得分：

1. 电机最终位置与主峰距离为 `d`：`d ≤ 3` 时基础得分为 `3000 − 200 × d`；`d > 3` 时该用例得 **0 分**。
2. 基础分再减去**搜索时间**、**振荡次数**与**冗余步数**：
   `case score = max(0, 3000 − 200×d − search_time − oscillation − redundant)`
3. 满足以下任一条件时该用例得 **0 分**：
   - 搜索步数 ≥ 200；
   - 振荡次数 ≥ 7；
   - 冗余步数 ≥ 200；
   - 用时超过 10 秒。

总分 = 所有用例得分之和（共 102 个用例）。0 分结果（停错峰、超振荡、超冗余、超查询）仍属"合法完成"，计入总分 0；协议错误或超时则整个作品判为失败（见错误类型）。

数据范围与说明

- 数组元素均为非负整数，取值在 `[0, 2^31-1]` 内。
- 数组长度固定为 **400 ~ 1000**。
- 所有用例**取自真实监控场景数据**，包括（但不限于）低照度噪声、运动物体、前景/背景等场景。

- 振荡、冗余步数、时间限制反映了当前业界 AF 算法在各类场景下的最差水平。
- 内存限制：512MB（平台级限制）。

样例

假设数组为 `[10, 14, 15, 16, 17, 12, 10, 8, 8, 9, 6, 5]`（主峰在下标 `4`，值 `17`，长度 12）。

程序与判题器的交互过程如下（行首 `→` 为判题器发给程序，`←` 为程序发出）：

```
→ 12
← 1
→ 14
← 4
→ 17
← 6
→ 10
← 5
→ 12
← 4
→ 17
← DONE
```

电机轨迹：`0 → 1 → 4 → 6 → 5 → 4`。

步	下标	移动	搜索步	搜索代价	搜索时间累计	振荡	冗余步数
1	1	0→1	1	1	2	0	0
2	4	1→4	1	3	6	0	0
3	6	4→6	1	2	9	0	2（最右 6 − 主峰 4）
4	5	6→5	1	1	11	1（右→左）	2
5	4	5→4	1	1	13	1	2

得分计算：

1. 停在主峰，`d=0`，基础分 3000；
2. 减去搜索时间 13：`3000 − 13`；
3. 减去振荡 1（<7，有效）：`3000 − 13 − 1`；
4. 减去冗余步数 2（<200，有效）：`3000 − 13 − 1 − 2`；
5. **最终得分：2984**。

错误类型

错误类型	说明	平台后果
编译错误	提交源码编译失败	整份作品失败
Invalid Output	协议错误：查询下标越界、输出非整数、单行超 64 字节、未发 DONE 提前退出、DONE 后仍有输出、退出码非 0、交互消息超过 201 条	整份作品失败
超出时间限制	单用例超过 10 秒未完成交互	整份作品失败
超出内存限制	程序内存超过 512MB	整份作品失败

注意：与"0 分结果"不同，协议错误（Invalid Output）或超时会导致**整份作品判为失败**，请务必严格遵守交互协议。